Par de puntos más cercano en $\mathbb{R}^3$: modificaciones al algoritmo Divide-and-Conquer

Documento técnico

26 de octubre de 2026

1. Planteamiento

Dado un conjunto P de n puntos en $\mathbb{R}^3$, se busca el par $p_i, p_j \in P$ que minimiza la distancia euclidiana

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2}.$$

La estrategia de *Divide-and-Conquer* (D&C) para el caso planar se conserva en esencia, pero la fase de fusión debe razonar sobre un *volumen* (una losa tridimensional) en lugar de una franja plana. Además, el número de vecinos candidatos por punto aumenta de 7, en el caso bidimensional, a 31 en el caso tridimensional.

2. Algoritmo modificado para 3D

Las modificaciones necesarias respecto al algoritmo clásico en $\mathbb{R}^2$ son las siguientes:

1. La división se realiza mediante un **plano mediano** en lugar de una recta vertical.
2. La región de fusión es una **losa de grosor** 2δ alrededor de dicho plano.
3. Los puntos de la losa se ordenan y recorren usando **dos coordenadas**, por ejemplo y y z, en lugar de una sola.
4. La cota de vecinos candidatos se amplía mediante un argumento de empaquetamiento en cajas tridimensionales.

```
Cercano3D(P):

1. Si |P| <= 3, resolver por fuerza bruta
   y regresar el par de distancia mínima.

2. Ordenar P por la coordenada x.
   (En una implementación eficiente, mantener
   listas preordenadas por x, y y z.)

3. Dividir:
   Trazar el plano mediano H: x = x_med,
   separando P en P_L y P_R.
```

```
4. Conquistar:
   Calcular recursivamente
   (p_L, delta_L) = Cercano3D(P_L)
   (p_R, delta_R) = Cercano3D(P_R)

5. Definir
   delta = min(delta_L, delta_R).

6. Fusionar:
   Formar la losa
   S = { p en P : |p.x - x_med| < delta }.

7. Ordenar S por y; en caso de empate, por z.

8. Para cada punto p en S:
      Comparar p con los siguientes puntos q de S
      mientras q.y - p.y < delta.
      Realizar a lo más 31 comparaciones por punto.
      Actualizar delta y el mejor par si
      d(p,q) < delta.

9. Regresar el mejor par entre:
      - el par encontrado en P_L,
      - el par encontrado en P_R,
      - el par encontrado en la losa S.
```

3. Región de fusión en 3D

En el algoritmo planar, la franja relevante tiene dimensiones $2\delta \times \delta$. En tres dimensiones, para un punto $p \in S$, todo candidato q con

$$d(p,q) < \delta \qquad \text{y} \qquad q_y \geq p_y$$

debe estar contenido en una caja de dimensiones

$$2\delta \times \delta \times 2\delta.$$

- La dimensión 2δ en x proviene de que ambos puntos deben estar dentro de la losa de grosor 2δ.
- La dimensión δ en y proviene del ordenamiento por y: solo se comparan puntos cuyo desplazamiento en y es menor que δ.
- La dimensión 2δ en z proviene de que la distancia euclidiana debe ser menor que δ.

4. ¿Cuántos puntos se comparan en la fusión?

Teorema 1. *En la fase de fusión del algoritmo de par de puntos más cercano en $\mathbb{R}^3$, cada punto de la losa debe compararse con a lo más 31 puntos siguientes en el orden por y.*

Demostración. Sea δ la menor distancia encontrada al resolver recursivamente las dos mitades. Por definición de δ, dentro de cada mitad no pueden existir dos puntos a distancia menor que δ.

Sea p un punto de la losa S. Todo punto candidato q tal que

$$q_y \geq p_y \qquad \text{y} \qquad d(p,q) < \delta$$

debe encontrarse dentro de una caja de dimensiones

$$2\delta \times \delta \times 2\delta.$$

Dividamos esta caja en cubos de lado $\delta/2$. El número de cubos es

$$4 \times 2 \times 4 = 32.$$

La diagonal de cada cubo es

$$\frac{\delta\sqrt{3}}{2} < \delta.$$

Por lo tanto, dos puntos distintos no pueden estar dentro del mismo cubo, pues su distancia sería menor que δ, contradiciendo la definición de δ.

En consecuencia, la caja contiene a lo más 32 puntos. Descontando el propio punto p, cada punto debe compararse con a lo más

$$32 - 1 = 31$$

puntos candidatos. □

Así, aunque la losa contenga m puntos, el trabajo de fusión es

$$O(31m) = O(m) = O(n).$$

Esto es análogo al caso planar, donde la cota clásica es de 7 vecinos por punto.

5. Complejidad en 3D

Con la cota constante de 31 vecinos, la recurrencia del algoritmo es

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n).$$

Por el teorema maestro,

$$T(n) = O(n \log n).$$

Por lo tanto, el algoritmo Divide-and-Conquer para el par de puntos más cercano en $\mathbb{R}^3$ tiene complejidad

$$\boxed{O(n \log n)}.$$

La constante oculta es mayor que en el caso bidimensional, ya que en la fusión se realizan hasta 31 comparaciones por punto en lugar de 7. Sin embargo, el orden asintótico no cambia.

Variante por proyección

Existe una variante más simple en la que, durante la fusión, se proyectan los puntos de la losa sobre el plano yz y se resuelve recursivamente un subproblema de par de puntos más cercano en $\mathbb{R}^2$.

En ese caso, la recurrencia es

$$T(n,3) = 2T\left(\frac{n}{2}, 3\right) + O(n \log n),$$

lo que produce

$$T(n,3) = O(n \log^2 n).$$

La versión con la cota de 31 vecinos evita esta recursión adicional y logra directamente $O(n \log n)$.

6. Generalización a d dimensiones

El esquema generaliza de forma directa a $\mathbb{R}^d$. Sin embargo, el algoritmo y sus constantes cambian.

- La división se realiza mediante un **hiperplano mediano** ortogonal a un eje, por ejemplo x_1.
- La franja o losa de fusión tiene grosor 2δ alrededor del hiperplano.
- Los puntos de la losa se ordenan lexicográficamente por las $d-1$ coordenadas restantes.
- La distancia utilizada es la norma euclidiana en $\mathbb{R}^d$.
- El número de comparaciones por punto en la fusión se convierte en

$$C(d) = 2^{2d-1} - 1.$$

La justificación es la misma que en 3D. La caja candidata tiene dimensiones

$$2\delta \times \delta \times \underbrace{2\delta \times \cdots \times 2\delta}_{d-2 \text{ veces}}.$$

Al dividirla en cubos de lado $\delta/2$, se obtienen

$$4 \times 2 \times 4^{d-2} = 2^{2d-1}$$

cubos. Como cada cubo puede contener a lo más un punto, el número máximo de vecinos candidatos por punto es

$$2^{2d-1} - 1.$$

En particular:

- Para $d = 2$:
$$2^3 - 1 = 7.$$
- Para $d = 3$:
$$2^5 - 1 = 31.$$

Dimensión	División	Candidatos por punto	Complejidad
2D	Recta vertical	7	$O(n \log n)$
3D	Plano mediano	31	$O(n \log n)$
d dimensiones	Hiperplano mediano	$2^{2d-1} - 1$	$O(n \log n)$ para d fija

Tabla 1: Comparación del algoritmo de par de puntos más cercano según la dimensión.

7. Tabla comparativa

8. ¿Qué ocurre si d crece?

Si d se considera parte de la entrada, y no una constante, la situación cambia sustancialmente:

- La constante
$$2^{2d-1} - 1$$
crece exponencialmente con d.
- Calcular una distancia euclidiana cuesta $O(d)$.
- Las formulaciones clásicas que resuelven recursivamente el problema en $d - 1$ dimensiones conducen a complejidades del tipo
$$O\left(n(\log n)^{d-1}\right).$$
- Para dimensionalidad alta se prefieren estructuras espaciales como *kd-trees*, rejillas espaciales, *locality-sensitive hashing* o algoritmos aproximados.

9. Respuestas directas

- **Modificaciones para 3D:** se usa un plano mediano en vez de una recta; la franja se convierte en una losa de grosor 2δ; el ordenamiento y barrido se realizan por y y z; y la cota de vecinos pasa a ser 31.
- **Puntos comparados en la fusión:** a lo más 31 vecinos siguientes por punto. Esto se debe al empaquetamiento: la caja $2\delta \times \delta \times 2\delta$ se divide en 32 cubos de lado $\delta/2$, cada uno con máximo un punto.
- **Complejidad en 3D:**
$$O(n \log n)$$
con la variante de vecinos constantes. La variante que resuelve un subproblema 2D en cada fusión tiene complejidad
$$O(n \log^2 n).$$
- **Escalamiento a d dimensiones:** la estructura del algoritmo se mantiene, pero la división usa hiperplanos, la fusión usa $d - 1$ coordenadas y los candidatos crecen a
$$2^{2d-1} - 1.$$

Para d fija, la complejidad sigue siendo $O(n \log n)$, pero con una constante exponencial en d. En formulaciones recursivas por proyección, la complejidad es

$$O\left(n(\log n)^{d-1}\right).$$

Referencias

- T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, sección 33.4: algoritmo de par de puntos más cercano, franja de 2δ y cota de 7 vecinos en el plano.
- S. Suri, *Closest Pair Problem*, University of California, Santa Barbara: generalización a d dimensiones y complejidad $O(n(\log n)^{d-1})$.
- S. Khuller y Y. Matias, análisis de algoritmos Divide-and-Conquer para el problema del par más cercano en dimensiones superiores.